

Spring Security Core Context & Internals

Q1. What are SecurityContext and SecurityContextHolder in Spring Security, and how do they participate in storing and propagating authentication information?

A: The **SecurityContext** is the object that holds the currently authenticated user's details (Authentication object). The **SecurityContextHolder** is the mechanism used to store the SecurityContext for the current thread of execution. Together, they ensure that the authenticated user's information is instantly available to any part of the application logic that needs to check security rules.

Q2. Explain the role of AuthenticationManager and ProviderManager in Spring Security. How does the authentication flow pass through them?

A: The **AuthenticationManager** is the main service API for authenticating a user. The flow passes through it when a user attempts to log in. The **ProviderManager** is the standard implementation, which delegates the actual verification to a list of configured **AuthenticationProviders** (e.g., a DAO provider for database checks or an LDAP provider). The first provider that successfully authenticates the user wins.

Q3. What is the purpose of the Spring Security filter chain, and how would you add, reorder, or customize a filter in that chain?

A: The **Spring Security Filter Chain** is a sequence of dedicated filters (like UsernamePasswordAuthenticationFilter) that handle security concerns for every HTTP request. You add a custom filter by creating a class that extends OncePerRequestFilter. You customize its position using `http.addFilterBefore()` or `http.addFilterAfter()` within the SecurityFilterChain bean configuration.

OAuth2 & Modern Auth Flows

Q4. How does Spring Security integrate with OAuth2 for authentication and authorization in modern applications?

A: Spring Security integrates with OAuth2 by providing specialized client and resource server configurations. For authentication, it allows your application to act as a client, redirecting users to an external identity provider (IdP) like Google. For authorization, it validates the tokens issued by the IdP and uses the user data within those tokens to apply security rules.

Q5. What is the Authorization Code grant type in OAuth2, and when would you choose it over other grant types?

A: The **Authorization Code grant type** is the standard, most secure flow used by client applications running in a browser. The user authenticates with an IdP, which returns a temporary code to your app. Your app then secretly exchanges this code for a long-lived access token. You choose it because it avoids exposing the sensitive access token directly in the browser's URL bar.

Q6. Explain, end-to-end, how OAuth2 authentication works in a Spring-based system (client, auth server, resource server).

A: The **Client** redirects the user to the **Authorization Server** (e.g., Google) to log in. The Auth Server sends back an authorization code to the Client. The Client secretly exchanges this code for a **JWT Access Token**. The Client then sends this Access Token in the header of API requests to the **Resource Server** (your Spring app), which validates the token to grant access.

JWT & Stateless Security

Q7. How can you integrate JWT (JSON Web Token) into a Spring Boot application to secure APIs?

A: To integrate JWT, you first configure Spring Security to be **stateless**. You then create a custom filter that intercepts every request, reads the JWT from the Authorization header, validates its signature, and extracts the user details and roles. These details are then placed into the SecurityContext for the duration of the request.

Q8. Your REST services must be completely stateless. Describe how you would implement JWT-based authentication using Spring Security, starting from login up to accessing protected endpoints.

A: During **login**, the application authenticates the user and generates a signed JWT. In the main security configuration, you disable session management (`sessionCreationPolicy(STATELESS)`). For subsequent requests, a **JWT filter** runs first, validates the token signature, and sets the Authentication object directly into the SecurityContextHolder. Access is then granted based on the roles found inside the token.

Method-Level & Fine-Grained Authorization

Q9. How do you enable and use method-level security in a Spring application (e.g., `@PreAuthorize`, `@PostAuthorize`), and what advantages does it give over URL-only security?

A: You enable method security by using the `@EnableMethodSecurity` annotation on a configuration class. You use `@PreAuthorize` to check permissions *before* a method runs or `@PostAuthorize` to check permissions *after* it runs. The advantage is **fine-grained control**, as security rules can check method arguments or returned values, which URL-based security cannot do.

Q10. What are the differences between method-level security and URL-based security in Spring Security? When do you prefer each?

A: **URL-based security** checks permissions based only on the request path and HTTP method (/admin/** or GET /users). **Method-level security** checks permissions based on the user's roles and the actual data in the request (`@PreAuthorize("hasRole('ADMIN')")`). You prefer URL-based security for general access control and method-level security for precise, business-logic-driven checks (e.g., "only the owner can delete this record").

Q11. How do you use Spring Expression Language (SpEL) in annotations like `@PreAuthorize` to implement fine-grained access control (e.g., row-level, ownership-based checks)?

A: You use **SpEL** inside `@PreAuthorize` to access method arguments or security principal details at runtime. For ownership checks, you might write `@PreAuthorize("authentication.name == #userId")`, where `#userId` references a parameter passed to the method. This allows for complex, dynamic security rules beyond simple role checks.

Q12. You have service methods that must only be executed by certain roles. How would you restrict them using method-level annotations in Spring Security?

A: I would use the `@PreAuthorize` annotation placed directly on the service method. I would write an expression like `@PreAuthorize("hasAnyRole('ROLE_ADMIN', 'ROLE_MANAGER')")`. This ensures that if the user calling the method lacks either the ADMIN or MANAGER role, the method execution is blocked before any business logic is executed.

Session Management & CSRF

Q13. How does Spring Security handle session management, and what options exist for limiting or controlling concurrent sessions per user?

A: Spring Security handles session management by integrating with the HttpSession mechanism and using a **SecurityContextPersistenceFilter** to manage the context. For concurrent sessions, you can configure the session policy to specify the maximum number of active sessions a single user can have (e.g., `.maximumSessions(1)`), and define whether subsequent logins should block or simply log out the previous session.

Q14. How is CSRF protection enabled and enforced in Spring Security, and how would you configure it for secure HTML form submission?

A: CSRF protection is **enabled by default** in Spring Security. It is enforced by a filter that requires a unique, secret **CSRF token** to be present in any modifying request (POST, PUT, DELETE). For HTML forms, you configure it by ensuring your form templates include the token, often retrieved via a Thymeleaf or JSP tag (e.g., `<input type="hidden" name="_csrf" value="..." />`).

Q15. In which scenarios would you change or relax CSRF behavior, and what compensating controls would you consider?

A: You relax CSRF behavior primarily when building **stateless REST APIs** that are consumed by non-browser clients (like mobile apps). In this scenario, you **disable** CSRF protection because the protection relies on sessions. The compensating control is the use of a secure, stateless token (like **JWT**) transmitted in the Authorization header.

Password Handling & Authentication Details

Q16. In your application, users' passwords must be stored securely. Which password encoding approach would you use in Spring Security, and why?

A: I would use the **BCrypt** encoding approach. BCrypt is highly recommended because it is a secure, modern, one-way hashing algorithm that is intentionally slow (making brute-force attacks harder). Crucially, BCrypt automatically handles the necessary **salting**, meaning I don't have to manage the salt value manually.

Q17. How do you configure a PasswordEncoder, and how is it used during login to verify user credentials?

A: You configure a PasswordEncoder by defining it as a **@Bean** in your configuration, typically by instantiating BCryptPasswordEncoder. During login, the configured DaoAuthenticationProvider retrieves the hashed password from the database. It then calls passwordEncoder.matches(rawPassword, hashedPassword) to verify the user's input against the stored hash.

Q18. What is digest authentication, and how does it differ from basic authentication in terms of security and usage?

A: **Basic authentication** sends credentials as clear, base64-encoded text (insecure). **Digest authentication** is a more secure, obsolete method where the client sends a hash of the password and a server-provided nonce. It differs by proving knowledge of the password without sending the password over the network, but it is rarely used today, replaced by TLS and token-based security.

Debugging & Testing Security

Q19. Users are being denied access despite having the correct roles. How would you systematically debug this in a Spring Security-enabled app (logs, debug filters, config check, authorities, etc.)?

A: I would first enable **Spring Security DEBUG logging** to trace the filter chain execution. I would verify the **Authentication** object to confirm the user's granted authorities are correct. If the authorities are missing, the UserDetailsService or token parsing is the issue. If the authorities are correct, the problem lies in the **SecurityFilterChain** or **@PreAuthorize** expression logic.

Q20. How do you test Spring Security configurations (authentication, authorization rules, filters) in automated tests? Which testing support does Spring provide?

A: Spring provides **@WebMvcTest** combined with the **MockMvc** utility for testing security. You use MockMvc to send mock HTTP requests while specifying security context details using annotations like **@WithMockUser** (to simulate a logged-in user) or **@WithUserDetails**. This allows you to assert that URL access is correctly granted or denied (e.g., checking for a 403 Forbidden status).

Configuration, Starters & Dependency Setup

Q21. What are the core building blocks/classes required to wire Spring Security into an application?

A: The core building blocks are the **SecurityFilterChain** (the main configuration entry point), a custom **UserDetailsService** (to load user details), and a **PasswordEncoder** (to handle password hashing). These are typically defined as beans and wired together within a class extending **WebSecurityConfigurerAdapter** (older versions) or by defining the **SecurityFilterChain** bean directly.

Q22. Is there a dedicated Spring Boot starter for Spring Security, or do you always add dependencies manually? How does this differ between a plain Spring MVC app and a Spring Boot app?

A: Yes, there is a dedicated starter: **spring-boot-starter-security**. In a Spring Boot app, adding this one dependency automatically sets up the default security configurations and filters. In a plain Spring MVC app, you must manually add the core security dependencies, configure the **FilterChainProxy** in your web deployment descriptor, and write more boilerplate Java configuration.

Q23. In a JWT-secured REST API, where and how do you plug a JWT parsing/verification filter into the Spring Security filter chain so that it works correctly with SecurityContext?

A: You plug the custom JWT filter into the chain by adding it **before** the standard **UsernamePasswordAuthenticationFilter**. The custom filter reads the token, verifies it, creates an **Authentication** object, and sets it into the **SecurityContextHolder**. Since it runs early, subsequent filters and application code will find the authenticated user details immediately.

Q24. How do you configure Spring Security when your app acts as an OAuth2 Resource Server versus when it acts as an Authorization Server?

A: When acting as a **Resource Server**, you configure Spring Security to validate the incoming access tokens (JWTs) using keys from the Authorization Server. When acting as an **Authorization Server** (a service that issues tokens), you configure endpoints for token issuance and management, including defining client details and redirect URIs.

Q25. How would you implement a custom UserDetailsService and UserDetails model when your user data model is complex (multiple roles, permissions, flags)?

A: I would implement a custom **UserDetailsService** (e.g., **CustomUserDetailsService**) to load the complex user data from the database. I would then create a custom **UserDetails** model

(e.g., CustomUserDetails) to represent the user. This model would store the complex attributes (like permissions or flags) and map them to Spring Security's expected authorities and properties.

Q26. How would you approach security in a multi-tenant system where the same Spring Boot app serves multiple clients with different auth/role rules?

A: I would implement a **Tenant Context** filter that identifies the tenant from the request (e.g., from a custom header or the URL subdomain) and stores the tenant ID in a ThreadLocal variable. Security rules would then check the user's role **and** the active tenant ID within custom **@PreAuthorize** expressions to enforce tenant-specific access control.

JAVA Interview Buddy